

TRIGGERS

1. Automatically Track Student Count :

```
1 • DROP DATABASE IF EXISTS CollegeStudentDB;
2 • CREATE DATABASE CollegeStudentDB;
3 • USE CollegeStudentDB;
4 • CREATE TABLE Students (
5     ID INT PRIMARY KEY,
6     Name VARCHAR(50),
7     Department VARCHAR(50)
8 );
9 • CREATE TABLE Total_Students (
10     Student_Count INT
11 );
12 • INSERT INTO Total_Students VALUES (0);
13 DELIMITER //
14 • CREATE TRIGGER count_new_student
15     AFTER INSERT ON Students
16     FOR EACH ROW
17 BEGIN
18     UPDATE Total_Students
19     SET Student_Count = Student_Count + 1;
20 END //
21 DELIMITER ;
22 • INSERT INTO Students VALUES
23     (201, 'Manya', 'BCA');
24 • INSERT INTO Students VALUES
25     (202, 'Navya', 'BCA');
26 • INSERT INTO Students VALUES
27     (203, 'Spoorthi', 'BCA');
28 • INSERT INTO Students VALUES
29     (204, 'Tanisha', 'BCA');
30 • INSERT INTO Students VALUES
31     (205, 'Neha', 'BCA');
32 • INSERT INTO Students VALUES
33     (206, 'Varshitha', 'BCA');
34 • SELECT * FROM Students;
35 • SELECT * FROM Total_Students;
```

OUTPUT:

Result Grid	Filter Rows:	Export:	Wrap Cell Contents:
Student_Count			
6			

Students 3	Total_Students 4
------------	------------------

#	Time	Action	Message
✓ 46	11:59:57	INSERT INTO Students VALUES (206, 'Varshitha', 'BCA')	1 row(s) affected
✓ 47	11:59:57	SELECT * FROM Students LIMIT 0, 2000	6 row(s) returned
✓ 48	11:59:57	SELECT * FROM Total_Students LIMIT 0, 2000	1 row(s) returned

2. Maintain an Automatic Salary Record :

Limit to 2000 rows
<pre> 1 • DROP DATABASE IF EXISTS OfficeSalaryDB; 2 • CREATE DATABASE OfficeSalaryDB; 3 • USE OfficeSalaryDB; 4 • CREATE TABLE Staff (5 Staff_ID INT PRIMARY KEY, 6 Staff_Name VARCHAR(50), 7 Job_Role VARCHAR(50), 8 Monthly_Salary DECIMAL(10,2) 9); 10 • CREATE TABLE Salary_History (11 History_ID INT AUTO_INCREMENT PRIMARY KEY, 12 Staff_ID INT, 13 Previous_Salary DECIMAL(10,2), 14 Updated_Salary DECIMAL(10,2), 15 Updated_On TIMESTAMP DEFAULT CURRENT_TIMESTAMP 16); 17 • INSERT INTO Staff VALUES 18 (101, 'Manya', 'Developer', 55000), 19 (102, 'Navya', 'Designer', 48000), 20 (103, 'Neha', 'Tester', 45000), 21 (104, 'Spoorthi', 'Analyst', 52000), 22 (105, 'Varshitha', 'Manager', 65000), 23 (106, 'Tanisha', 'Developer', 58000); </pre>

```

25 DELIMITER //
26 • CREATE TRIGGER record_salary_change
27 AFTER UPDATE ON Staff
28 FOR EACH ROW
29 BEGIN
30     IF OLD.Monthly_Salary <> NEW.Monthly_Salary THEN
31         INSERT INTO Salary_History
32             (Staff_ID, Previous_Salary, Updated_Salary)
33             VALUES
34             (OLD.Staff_ID, OLD.Monthly_Salary, NEW.Monthly_Salary);
35     END IF;
36 END //
37 DELIMITER ;
38 • UPDATE Staff
39     SET Monthly_Salary = 60000
40     WHERE Staff_ID = 101;
41 • UPDATE Staff
42     SET Monthly_Salary = 50000
43     WHERE Staff_ID = 102;
44 • UPDATE Staff
45     SET Monthly_Salary = 48000
46     WHERE Staff_ID = 103;
47 • UPDATE Staff
48     SET Monthly_Salary = 57000
49     WHERE Staff_ID = 104;
50 • UPDATE Staff
51     SET Monthly_Salary = 70000
52     WHERE Staff_ID = 105;
53 • UPDATE Staff
54     SET Monthly_Salary = 63000
55     WHERE Staff_ID = 106;
56 • SELECT * FROM Staff;
57 • SELECT * FROM Salary_History;

```

OUTPUT:

History_ID	Staff_ID	Previous_Salary	Updated_Salary	Updated_On
1	101	55000.00	60000.00	2026-09-29 12:13:55
2	102	48000.00	50000.00	2026-09-29 12:13:55
3	103	45000.00	48000.00	2026-09-29 12:13:55
4	104	52000.00	57000.00	2026-09-29 12:13:55
5	105	65000.00	70000.00	2026-09-29 12:13:55
6	106	58000.00	63000.00	2026-09-29 12:13:55

#	Time	Action	Message
61	12:13:55	UPDATE Staff SET Monthly_Salary = 48000 WHERE Staff_ID = 103	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
62	12:13:55	UPDATE Staff SET Monthly_Salary = 57000 WHERE Staff_ID = 104	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
63	12:13:55	UPDATE Staff SET Monthly_Salary = 70000 WHERE Staff_ID = 105	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
64	12:13:55	UPDATE Staff SET Monthly_Salary = 63000 WHERE Staff_ID = 106	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
65	12:13:55	SELECT * FROM Staff LIMIT 0, 2000	6 row(s) returned
66	12:13:55	SELECT * FROM Salary_History LIMIT 0, 2000	6 row(s) returned

3.Prevent Deletion of Important Staff:

```
1 • DROP DATABASE IF EXISTS CompanyStaffDB;
2 CREATE DATABASE CompanyStaffDB;
3 • USE CompanyStaffDB;
4 • CREATE TABLE Staff (
5     Staff_ID INT PRIMARY KEY,
6     Staff_Name VARCHAR(50),
7     Department VARCHAR(50),
8     Salary DECIMAL(10,2)
9 );
10 • CREATE TABLE Staff_Delete_Log (
11     Log_ID INT AUTO_INCREMENT PRIMARY KEY,
12     Staff_ID INT,
13     Staff_Name VARCHAR(50),
14     Delete_Time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
15 );
16 • INSERT INTO Staff VALUES
17     (201, 'Manya', 'IT', 55000),
18     (202, 'Navya', 'HR', 48000),
19     (203, 'Neha', 'Management', 75000),
20     (204, 'Spoorthi', 'Finance', 60000),
21     (205, 'Varshitha', 'Sales', 52000),
22     (206, 'Tanisha', 'Management', 70000);
23 DELIMITER //

23 DELIMITER //
24 • CREATE TRIGGER stop_management_delete
25 BEFORE DELETE ON Staff
26 FOR EACH ROW
27 BEGIN
28     IF OLD.Department = 'Management' THEN
29         SIGNAL SQLSTATE '45000'
30         SET MESSAGE_TEXT = 'Management staff cannot be deleted';
31     END IF;
32 END //
33 • CREATE TRIGGER save_deleted_staff
34 AFTER DELETE ON Staff
35 FOR EACH ROW
36 BEGIN
37     INSERT INTO Staff_Delete_Log
38     (Staff_ID, Staff_Name)
39     VALUES
40     (OLD.Staff_ID, OLD.Staff_Name);
41 END //
42 DELIMITER ;
43 • DELETE FROM Staff
44 WHERE Staff_ID = 201;
```

```

41      END //
42      DELIMITER ;
43      DELETE FROM Staff
44      WHERE Staff_ID = 201;
45      SELECT * FROM Staff;
46      SELECT * FROM Staff_Delete_Log;
47      DELETE FROM Staff
48      WHERE Staff_ID = 203;

```

OUTPUT:

Result Grid				
Filter Rows:				
Edit:				
Export/Import:				
Wrap Cell Content:				
	Log_ID	Staff_ID	Staff_Name	Delete_Time
▶	1	201	Manya	2026-09-29 12:26:14
✱	NULL	NULL	NULL	NULL